Control digital de un péndulo invertido lineal con ESP32 y motor a pasos

Control Avanzado · Unidad 2: Controladores digitales

Grupo MTR09A · Equipo 8

Integrantes: [nombres de los integrantes]

1. Objetivo

Estabilizar un péndulo invertido sobre un carro en la posición vertical mediante un controlador PD digital diseñado por lugar geométrico de las raíces en el plano z, e implementar en un ESP32 exactamente la ecuación en diferencias obtenida en MATLAB. El proyecto continúa el péndulo del cuatrimestre anterior, controlado con un PD analógico con amplificadores operacionales.

2. Descripción del sistema

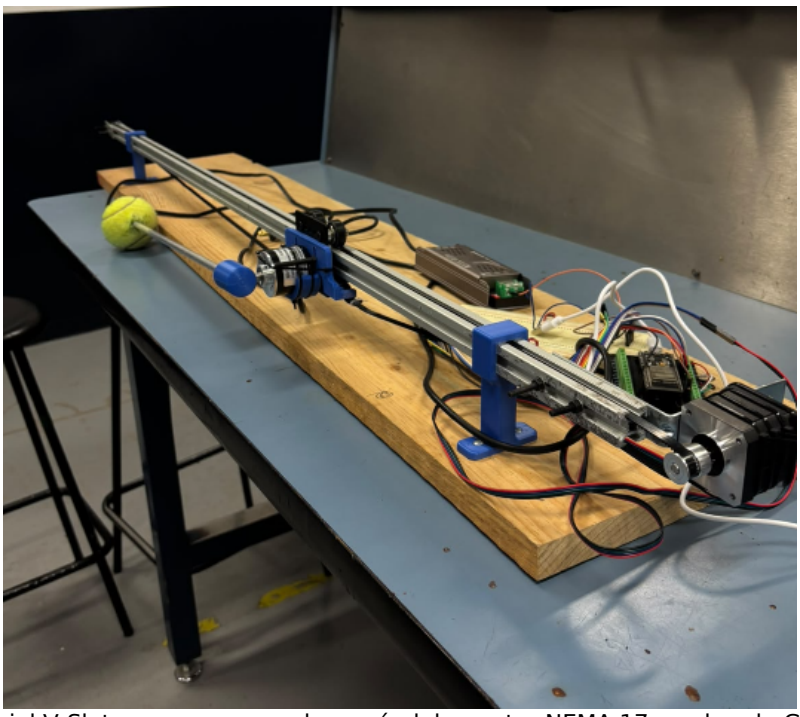


Figura 1. Prototipo: riel V-Slot, carro con encoder y péndulo, motor NEMA 17 con banda GT2, driver DRV8825 y ESP32.

Elemento	Especificación
Microcontrolador	ESP32 (NodeMCU-32S), doble núcleo a 240 MHz, framework Arduino en PlatformIO
Actuador	Motor a pasos NEMA 17 17HS4401, driver DRV8825 (≈1.3 A), micropasos 1/8
Transmisión	Banda GT2, polea de 20 dientes (40 mm/vuelta): 1600 pasos/vuelta = 40 000 pasos/m
Sensor	Encoder incremental LPD3806-600BM, 600 PPR × 4 = 2400 cuentas/vuelta (0.15°/cuenta)
Parámetros	g = 9.81 m/s², l = 0.342 m (pivote a centro de masa), recorrido útil del carro 0.88 m

3. Modelo de la planta

El motor de CD del cuatrimestre anterior ya no estaba disponible y se sustituyó por un motor a pasos. Esto cambia la entrada de la planta: el motor de CD impone una **fuerza** y el movimiento

del carro depende de su masa y de la reacción del péndulo; el motor a pasos impone directamente el **movimiento** del carro mientras no pierda pasos. Por eso la entrada pasa a ser la **aceleración del carro a** [m/s²]. De la ecuación linealizada del péndulo alrededor de la vertical:

$$l\cdot\theta'' = g\cdot\theta - a \quad \Rightarrow \quad G(s) = \Theta(s)/A(s) = -1 / (l\cdot s^2 - g)$$

Es el caso límite del modelo con fuerza cuando m/M → 0: el mismo sistema físico, con el mismo orden y el mismo polo inestable en $s = +\sqrt{(g/l)} = +5.36$ rad/s, por lo que la metodología de diseño de la clase se mantiene. El supuesto clave es que el motor no pierda pasos; por eso la aceleración comandada siempre se satura (sección 6).

4. Diseño del controlador digital (MATLAB)

Paso 1. Tiempo de muestreo. T = 0.01 s (100 Hz). La frecuencia de muestreo (628 rad/s) es unas 50 veces la frecuencia natural del lazo cerrado (12.3 rad/s) y unas 117 veces el polo inestable, de modo que el retenedor agrega poco atraso de fase.

Paso 2. Discretización con retenedor de orden cero: $G(z) = (1 - z^{-1})\cdot Z\{G(s)/s\}$

$$G(z) = 1.4623\times10^{-4}\cdot(z + 1) / (z^2 - 2.002869\cdot z + 1)$$

Paso 3. Lugar geométrico de las raíces en z. Se propuso un PD digital con un polo en el origen y un cero en zc:

$$D(z) = K\cdot(z - zc) / z, \quad K = 635.94, \quad zc = 0.91875$$

El lazo cerrado es estable (todos los polos dentro del círculo unitario) para **121 < K < 6822**. Con el valor elegido, los polos de lazo cerrado quedan en $z = 0.903 \pm j0.065$ (equivalentes a $\omega n \approx 12.3$ rad/s, $\zeta \approx 0.81$) y un polo rápido en z = 0.104. K queda lejos de ambos límites del margen.

Paso 4. Ecuación en diferencias. Con e[k] = 0 − θ[k]:

$$D(z) = K\cdot(1 - zc\cdot z^{-1}) \;\Rightarrow\; U(z) = K\cdot E(z) - K\cdot zc\cdot z^{-1}E(z)$$

$$\mathbf{u[k] = 635.94\cdot e[k] - 584.27\cdot e[k-1]} \quad (K\cdot zc = 635.94 \times 0.91875 = 584.27)$$

Como el signo negativo de la planta se absorbió en el diseño, la aceleración física del carro es a[k] = −u[k], es decir a[k] = 635.94·θ[k] − 584.27·θ[k−1]. La acción derivativa ya está contenida en el término e[k−1]; no se calcula una derivada por separado.

5. Implementación en el ESP32

Muestreo determinista. El controlador corre en una tarea de FreeRTOS fijada al núcleo 0, con prioridad alta, que se repite cada 10 ms exactos con vTaskDelayUntil. En cada periodo: lee θ, calcula el error, aplica la ecuación en diferencias, satura, manda al motor y guarda e[k] como e[k−1]. La telemetría y los comandos por puerto serie corren en el núcleo 1 para no alterar el periodo, que se mide en cada ciclo.

Sensor. El encoder se lee con el contador de pulsos de hardware (PCNT) con filtro activado, así no se pierden cuentas. El cero se toma con el péndulo colgando y quieto; la vertical es esa posición más 1200 cuentas (media vuelta), por lo que no depende de alinearlo a ojo.

Actuador. El controlador entrega aceleración, pero el motor se maneja por velocidad. La conversión es explícita: v[k] = v[k−1] + a[k]·T, que se convierte a pasos/s (× 40 000). En cada periodo se pide al motor alcanzar v[k] con aceleración constante |a[k]|, de modo que la aceleración se mantiene durante T, igual que el retenedor de orden cero supuesto en el diseño. Los pulsos de paso los genera el hardware (librería FastAccelStepper).

Condición inicial. Al arrancar se usa e[k−1] = e[k] y v = 0. Con e[k−1] = 0, la primera muestra pediría ≈111 m/s² con el péndulo a 10°; así pide los ≈9 m/s² esperados por el diseño.

La Tabla 1 relaciona cada resultado del diseño con el código; la Figura 2 muestra la función del controlador.

Resultado del diseño	Dónde está en el código
T = 0.01 s	config.h: const float T = 0.01; main.cpp: vTaskDelayUntil cada 10 ticks de 1 ms
K = 635.94 (coeficiente de e[k])	config.h: COEF_ERROR_ACTUAL = 635.94
K·zc = 584.27 (coeficiente de e[k−1])	config.h: COEF_ERROR_ANTERIOR = 584.27
e[k] = r − θ[k]	control.cpp: errorActual = thetaReferencia − theta
u[k] = 635.94 e[k] − 584.27 e[k−1]	control.cpp: u = COEF_ERROR_ACTUAL·errorActual − COEF_ERROR_ANTERIOR·errorAnterior
a = −u	control.cpp: aceleracionComando = −u
Retardo z^{-1}	control.cpp: errorAnterior = errorActual (al final de cada periodo)

Tabla 1. Correspondencia entre el diseño en MATLAB y el programa del ESP32.

```
void controlCalcular(float theta, float posicionCarro) {
  // Referencia: 0 en el diseño original; el lazo externo (sección 7) la ajusta
  errorActual = thetaReferencia - theta;                          // e[k]
  // Ecuación en diferencias (paso 4): u[k] = 635.94 e[k] - 584.27 e[k-1]
  float u = COEF_ERROR_ACTUAL * errorActual - COEF_ERROR_ANTERIOR * errorAnterior;
  aceleracionComando = -u;                                        // a = -u
  aceleracionComando = constrain(aceleracionComando, -ACELERACION_MAXIMA, ACELERACION_MAXIMA);
  velocidadComando = velocidadComando + aceleracionComando * T;        // v[k] = v[k-1] + a[k]·T
  velocidadComando = constrain(velocidadComando, -VELOCIDAD_MAXIMA, VELOCIDAD_MAXIMA);
  errorAnterior = errorActual;                                    // e[k] -> e[k-1]
}
```

Figura 2. Controlador implementado (control.cpp, versión resumida).

6. Saturaciones y seguridad

En la prueba de banco el motor no perdió pasos hasta 7 m/s² (con 8 m/s² ya los perdía). Para conservar margen, la aceleración se satura en **5.5 m/s²** y la velocidad en **0.4 m/s**. Con la ecuación de diseño la saturación se alcanza a partir de unos 6° de inclinación; la simulación no lineal con estos límites es estable desde 2° hasta 10°. Protecciones: el control se apaga si |θ| > 30° (el péndulo ya cayó) o si el carro se aleja más de 0.44 m del centro (límite por software con la cuenta de pasos), y solo arranca por comando con el carro centrado y |θ| < 5°.

7. Lazo externo de posición (cascada)

El controlador de la clase solo mide el ángulo, por lo que el péndulo se equilibra pero el carro deriva lentamente hasta el extremo del riel (en simulación, ≈0.7 m en 3 s). Para resolverlo se agregó un lazo externo lento que genera la referencia de ángulo del lazo interno; **el lazo interno es exactamente el diseño anterior, sin cambios**:

$$\theta_{ref} = -(0.05 \cdot x + 0.10 \cdot v), \quad \text{limitada a } \pm 3°, \quad e[k] = \theta_{ref} - \theta[k]$$

Si el carro se aleja del centro, el lazo externo pide una pequeña inclinación hacia el centro; para sostenerla, el lazo interno acelera el carro de regreso. Las ganancias se eligieron como $g \cdot 0.05 = \omega^2$ ($\omega \approx 0.7$ rad/s, unas 17 veces más lento que el lazo interno, para que no interfieran) y $g \cdot 0.10 = 2\zeta\omega$ ($\zeta \approx 0.7$). En la primera prueba la referencia cambiaba en cada muestra y la parte derivativa del lazo interno la amplificaba hasta saturar el motor; se corrigió con un filtro de primer orden de

la referencia ($\tau = 0.5$ s).

8. Pruebas y resultados

Cada prueba se hizo en un programa separado que reutiliza los mismos módulos del programa final:

Prueba	Criterio	Resultado
1. Encoder en banco	1 vuelta = 2400 cuentas, 90° = 600, regresa a la marca	2400, 600 y 0 cuentas al regresar: aprobada
2. Motor en banco	Aceleración máxima sin perder pasos	7 m/s² a 0.5 m/s sin pérdidas; con 8 m/s² pierde pasos
3. Controlador sin motor	Periodo de 10 ms; aceleración coherente con la ecuación	Periodo constante de 10 ms. Péndulo quieto a 3°: $a = (635.94 - 584.27)\cdot\theta = 51.67\cdot 0.0524 = 2.71$ m/s²; medido: 2.71 m/s²
4. Verificación de signo	El carro empuja hacia el lado al que cae el péndulo	Correcto en ambos sentidos: aprobada
5. Lazo cerrado (diseño de la clase)	El péndulo se mantiene vertical	Se equilibra sin vibración hasta que la deriva lleva el carro al límite del riel
5b. Lazo cerrado + cascada	Equilibrio sostenido	Más de 20 s equilibrado, carro prácticamente al centro, θref suave

Tabla 2. Pruebas por fases.

La prueba 3 confirma que el ESP32 ejecuta exactamente la ecuación de diseño: el valor calculado a mano con los coeficientes de MATLAB coincide con el medido.

9. Conclusiones

Se implementó en el ESP32 el controlador PD digital diseñado en MATLAB por lugar de las raíces en z, con la misma ecuación en diferencias, el mismo periodo de muestreo y un muestreo determinista verificado. El cambio a motor a pasos se justificó tomando la aceleración del carro como entrada, lo que conserva la estructura de la planta. El diseño estabilizó el péndulo en la vertical; la única limitación observada fue la deriva del carro, propia de controlar solo el ángulo, que se resolvió con un lazo externo de posición en cascada sin modificar el controlador de la clase.

Como trabajo adicional se implementó un arranque desde la posición colgando (swing-up por energía), que no forma parte del diseño de la clase. Con la fricción medida en el eje del encoder, la simulación indica que el carro no puede aportar suficiente energía dentro de su recorrido y con las saturaciones de seguridad; queda como trabajo futuro reducir la fricción del pivote.